

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

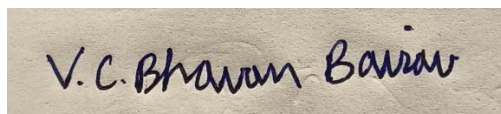
8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I V.C.BHAVAN BAIRAV (192511369) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1) A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF.

SCENARIO UNDERSTANDING:

The university stores student and course details in a single table. This causes data redundancy and anomalies, so the database must be normalized up to 3NF.

SOLUTION APPROACH:

The database is divided into separate tables such as Student, Department, Course, and Enrollment. This removes redundancy and improves data consistency.

```
mysql> use ssp;
Database changed
mysql> CREATE TABLE Student (
  ->     StudentID INT PRIMARY KEY,
  ->     StudentName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Faculty (
  ->     FacultyID VARCHAR(10) PRIMARY KEY,
  ->     FacultyName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Course (
  ->     CourseID VARCHAR(10) PRIMARY KEY,
  ->     CourseName VARCHAR(50),
  ->     FacultyID VARCHAR(10),
  ->     FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Enrollment (
  ->     StudentID INT,
  ->     CourseID VARCHAR(10),
  ->     PRIMARY KEY (StudentID, CourseID),
  ->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
  ->     FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
  -> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO Student VALUES
    -> (101,'Aryan'),
    -> (102,'Rahul');
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Faculty VALUES
    -> ('F101','Dr. Kumar'),
    -> ('F102','Dr. Meena');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Course VALUES
    -> ('C101','DBMS','F101'),
    -> ('C102','Java','F102');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Enrollment VALUES
    -> (101,'C101'),
    -> (101,'C102'),
    -> (102,'C101');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Student;
+-----+-----+
| StudentID | StudentName |
+-----+-----+
|      101 | Aryan      |
|      102 | Rahul      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Faculty;
+-----+-----+
| FacultyID | FacultyName |
+-----+-----+
| F101      | Dr. Kumar   |
| F102      | Dr. Meena   |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Course;
+-----+-----+-----+
| CourseID | CourseName | FacultyID |
+-----+-----+-----+
| C101     | DBMS       | F101      |
| C102     | Java       | F102      |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollment;
+-----+-----+
| StudentID | CourseID |
+-----+-----+
|      101 | C101     |
|      102 | C101     |
|      101 | C102     |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

RESULT AND JUSTIFICATION:

The normalized database stores data efficiently, reduces duplication, and maintains relationships using primary and foreign keys.

2) A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach.

SCENARIO UNDERSTANDING:

The company allows employees to work in multiple departments. The database should correctly represent this relationship.

SOLUTION APPROACH:

A junction table is created to connect employees and departments. SQL joins and grouping are used to retrieve employees working in multiple departments.

```
mysql>
mysql> create database j;
Query OK, 1 row affected (0.01 sec)

mysql> use j;
Database changed
mysql> CREATE TABLE Employee (
  ->     Emp_ID INT PRIMARY KEY,
  ->     Emp_Name VARCHAR(50),
  ->     Designation VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO Employee VALUES
  -> (101,'Arun','Manager'),
  -> (102,'Priya','Developer'),
  -> (103,'Rahul','Analyst'),
  -> (104,'Meena','Tester');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE TABLE Department (
  ->     Dept_ID INT PRIMARY KEY,
  ->     Dept_Name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Department VALUES
  -> (1,'HR'),
  -> (2,'IT'),
  -> (3,'Finance');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```

mysql>
mysql> CREATE TABLE Employee_Department (
  ->     Emp_ID INT,
  ->     Dept_ID INT,
  ->     PRIMARY KEY (Emp_ID, Dept_ID),
  ->     FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID),
  ->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
  -> );
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> INSERT INTO Employee_Department VALUES
  -> (101,1),
  -> (101,2),
  -> (102,2),
  -> (103,2),
  -> (103,3),
  -> (104,1);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql>
mysql> SELECT
  ->     E.Emp_ID,
  ->     E.Emp_Name,
  ->     COUNT(ED.Dept_ID) AS Total_Departments
  -> FROM Employee E
  -> INNER JOIN Employee_Department ED
  -> ON E.Emp_ID = ED.Emp_ID
  -> GROUP BY E.Emp_ID, E.Emp_Name
  -> HAVING COUNT(ED.Dept_ID) > 1;
+-----+-----+-----+
| Emp_ID | Emp_Name | Total_Departments |
+-----+-----+-----+
| 101 | Arun | 2 |
| 103 | Rahul | 2 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>

```

RESULT AND JUSTIFICATION:

The solution accurately represents the many-to-many relationship and retrieves the required employee information efficiently.

3) Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

SCENARIO UNDERSTANDING:

The given relation contains multiple functional dependencies that may lead to redundancy. The relation must be checked for BCNF.

SOLUTION APPROACH:

The relation is analyzed and decomposed into separate tables to satisfy BCNF and eliminate redundancy.

```
mysql> CREATE TABLE Department (  
-> Dept_ID VARCHAR(5) PRIMARY KEY,  
-> Dept_Name VARCHAR(50),  
-> Manager VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql>  
mysql> INSERT INTO Department VALUES  
-> ('D01','CSE','Dr. Kumar'),  
-> ('D02','IT','Dr. Ravi');  
Query OK, 2 rows affected (0.03 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> CREATE TABLE Employee (  
-> Emp_ID INT PRIMARY KEY,  
-> Emp_Name VARCHAR(50),  
-> Dept_ID VARCHAR(5),  
-> FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
-> );  
Query OK, 0 rows affected (0.08 sec)  
  
mysql>  
mysql> INSERT INTO Employee VALUES  
-> (101,'Arun','D01'),  
-> (102,'Priya','D02'),  
-> (103,'Rahul','D01'),  
-> (104,'Meena','D02');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql>
mysql> SELECT
->     E.Emp_ID,
->     E.Emp_Name,
->     D.Dept_Name,
->     D.Manager
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.Dept_ID = D.Dept_ID;
```

Emp_ID	Emp_Name	Dept_Name	Manager
101	Arun	CSE	Dr. Kumar
103	Rahul	CSE	Dr. Kumar
102	Priya	IT	Dr. Ravi
104	Meena	IT	Dr. Ravi

```
4 rows in set (0.01 sec)

mysql> |
```

RESULT AND JUSTIFICATION:

The decomposed tables improve data integrity, remove redundancy, and satisfy Boyce–Codd Normal Form.

4) A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

SCENARIO UNDERSTANDING:

The relation contains multivalued attributes that cause duplicate data and inefficient storage.

SOLUTION APPROACH:

Independent multivalued attributes are separated into different tables to satisfy Fourth Normal Form (4NF).

```
mysql> CREATE TABLE Student (  
    ->     Student_ID INT PRIMARY KEY,  
    ->     Student_Name VARCHAR(50)  
    -> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> INSERT INTO Student VALUES  
    -> (101,'Arun'),  
    -> (102,'Priya'),  
    -> (103,'Rahul');  
Query OK, 3 rows affected (0.02 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> CREATE TABLE Hobby (  
    ->     Hobby_ID INT PRIMARY KEY,  
    ->     Hobby_Name VARCHAR(50)  
    -> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql>  
mysql> INSERT INTO Hobby VALUES  
    -> (1,'Cricket'),  
    -> (2,'Music'),  
    -> (3,'Reading');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> CREATE TABLE Language (  
    ->     Language_ID INT PRIMARY KEY,  
    ->     Language_Name VARCHAR(50)  
    -> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql>  
mysql> INSERT INTO Language VALUES  
    -> (1,'English'),  
    -> (2,'Tamil'),  
    -> (3,'Hindi');  
Query OK, 3 rows affected (0.00 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```

mysql>
mysql> CREATE TABLE Student_Hobby (
  ->     Student_ID INT,
  ->     Hobby_ID INT,
  ->     PRIMARY KEY (Student_ID, Hobby_ID),
  ->     FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
  ->     FOREIGN KEY (Hobby_ID) REFERENCES Hobby(Hobby_ID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO Student_Hobby VALUES
  -> (101,1),
  -> (101,2),
  -> (102,3),
  -> (103,1);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> CREATE TABLE Student_Language (
  ->     Student_ID INT,
  ->     Language_ID INT,
  ->     PRIMARY KEY (Student_ID, Language_ID),
  ->     FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
  ->     FOREIGN KEY (Language_ID) REFERENCES Language(Language_ID)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO Student_Language VALUES
  -> (101,1),
  -> (101,2),
  -> (102,1),
  -> (103,3);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

```

```

mysql>
mysql> SELECT
  ->     S.Student_Name,
  ->     H.Hobby_Name,
  ->     L.Language_Name
  -> FROM Student S
  -> LEFT JOIN Student_Hobby SH
  -> ON S.Student_ID = SH.Student_ID
  -> LEFT JOIN Hobby H
  -> ON SH.Hobby_ID = H.Hobby_ID
  -> LEFT JOIN Student_Language SL
  -> ON S.Student_ID = SL.Student_ID
  -> LEFT JOIN Language L
  -> ON SL.Language_ID = L.Language_ID;

```

Student_Name	Hobby_Name	Language_Name
Arun	Cricket	English
Arun	Cricket	Tamil
Arun	Music	English
Arun	Music	Tamil
Priya	Reading	English
Rahul	Cricket	Hindi

```

6 rows in set (0.01 sec)

mysql> |

```

RESULT AND JUSTIFICATION:

The decomposition removes multivalued dependencies and improves database consistency and storage efficiency.

5) A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

SCENARIO UNDERSTANDING:

The relation contains join dependencies that may create redundant data and complex relationships.

SOLUTION APPROACH:

The database is decomposed into smaller related tables to eliminate join dependencies and achieve Fifth Normal Form (5NF).

```
mysql> CREATE DATABASE SupplyDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE SupplyDB;
Database changed

mysql>
mysql> CREATE TABLE Supplier (
    ->     Supplier_ID INT PRIMARY KEY,
    ->     Supplier_Name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO Supplier VALUES
    -> (1, 'ABC Suppliers'),
    -> (2, 'XYZ Suppliers');
Query OK, 2 rows affected (0.03 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> CREATE TABLE Product (
    ->     Product_ID INT PRIMARY KEY,
    ->     Product_Name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> INSERT INTO Product VALUES
    -> (101, 'Laptop'),
    -> (102, 'Printer');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> CREATE TABLE Project (
    ->     Project_ID INT PRIMARY KEY,
    ->     Project_Name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO Project VALUES
    -> (201, 'Project A'),
    -> (202, 'Project B');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```

mysql>
mysql> CREATE TABLE Supplier_Product (
  ->     Supplier_ID INT,
  ->     Product_ID INT,
  ->     PRIMARY KEY (Supplier_ID, Product_ID),
  ->     FOREIGN KEY (Supplier_ID) REFERENCES Supplier(Supplier_ID),
  ->     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO Supplier_Product VALUES
  -> (1,101),
  -> (1,102),
  -> (2,102);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE TABLE Supplier_Project (
  ->     Supplier_ID INT,
  ->     Project_ID INT,
  ->     PRIMARY KEY (Supplier_ID, Project_ID),
  ->     FOREIGN KEY (Supplier_ID) REFERENCES Supplier(Supplier_ID),
  ->     FOREIGN KEY (Project_ID) REFERENCES Project(Project_ID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO Supplier_Project VALUES
  -> (1,201),
  -> (1,202),
  -> (2,202);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE TABLE Product_Project (
  ->     Product_ID INT,
  ->     Project_ID INT,
  ->     PRIMARY KEY (Product_ID, Project_ID),
  ->     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID),
  ->     FOREIGN KEY (Project_ID) REFERENCES Project(Project_ID)
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> INSERT INTO Product_Project VALUES
  -> (101,201),
  -> (102,201),
  -> (102,202);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

```

```

mysql>
mysql> SELECT
  ->     S.Supplier_Name,
  ->     P.Product_Name,
  ->     PR.Project_Name
  -> FROM Supplier_Product SP
  -> INNER JOIN Supplier S
  -> ON SP.Supplier_ID = S.Supplier_ID
  -> INNER JOIN Product P
  -> ON SP.Product_ID = P.Product_ID
  -> INNER JOIN Product_Project PP
  -> ON P.Product_ID = PP.Product_ID
  -> INNER JOIN Project PR
  -> ON PP.Project_ID = PR.Project_ID;

```

Supplier_Name	Product_Name	Project_Name
ABC Suppliers	Printer	Project A
XYZ Suppliers	Printer	Project A
ABC Suppliers	Laptop	Project A
ABC Suppliers	Printer	Project B
XYZ Suppliers	Printer	Project B

5 rows in set (0.03 sec)

```
mysql> |
```

RESULT AND JUSTIFICATION:

The database becomes easier to maintain, reduces redundancy, and preserves all relationships without data loss.

6) Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

SCENARIO UNDERSTANDING:

The database requires rules to ensure only valid and consistent data is stored.

SOLUTION APPROACH:

Primary keys, foreign keys, unique, check, and not-null constraints are implemented to enforce data integrity.

```
mysql> create database r;
Query OK, 1 row affected (0.04 sec)

mysql> use r;
Database changed
mysql> CREATE TABLE Department (
    ->     Dept_ID INT PRIMARY KEY,
    ->     Dept_Name VARCHAR(50) UNIQUE NOT NULL
    -> );
Query OK, 0 rows affected (0.13 sec)

mysql>
mysql> INSERT INTO Department VALUES
    -> (101,'CSE'),
    -> (102,'ECE'),
    -> (103,'IT');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE TABLE Student (
    ->     Student_ID INT PRIMARY KEY,
    ->     Student_Name VARCHAR(50) NOT NULL,
    ->     Age INT CHECK (Age >= 18),
    ->     Gender CHAR(1) CHECK (Gender IN ('M','F')),
    ->     Marks INT CHECK (Marks BETWEEN 0 AND 100),
    ->     Dept_ID INT NOT NULL,
    ->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> INSERT INTO Student VALUES
    -> (1,'Arun',20,'M',90,101),
    -> (2,'Priya',19,'F',85,102),
    -> (3,'Rahul',21,'M',78,101),
    -> (4,'Meena',20,'F',92,103);
Query OK, 4 rows affected (0.01 sec)
```

```

mysql>
mysql> SELECT
->     S.Student_ID,
->     S.Student_Name,
->     S.Age,
->     S.Gender,
->     S.Marks,
->     D.Dept_Name
-> FROM Student S
-> INNER JOIN Department D
-> ON S.Dept_ID = D.Dept_ID;
+-----+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Age | Gender | Marks | Dept_Name |
+-----+-----+-----+-----+-----+-----+
|          1 | Arun         | 20  | M      | 90    | CSE       |
|          3 | Rahul        | 21  | M      | 78    | CSE       |
|          2 | Priya        | 19  | F      | 85    | ECE       |
|          4 | Meena        | 20  | F      | 92    | IT        |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.03 sec)

mysql>

```

RESULT AND JUSTIFICATION:

The constraints prevent invalid data, maintain consistency, and improve the reliability of the database.

7) A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

SCENARIO UNDERSTANDING:

The database must automatically perform specific actions whenever data is inserted, updated, or deleted.

SOLUTION APPROACH:

A trigger is created to enforce business rules and automatically record or validate changes.

```
mysql> CREATE DATABASE BankDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE BankDB;
Database changed
mysql>
mysql> CREATE TABLE Customer (
  ->     Customer_ID INT PRIMARY KEY,
  ->     Customer_Name VARCHAR(50),
  ->     Balance DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> INSERT INTO Customer VALUES
  -> (101,'Arun',5000),
  -> (102,'Priya',8000),
  -> (103,'Rahul',12000);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE TABLE Transaction_History (
  ->     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
  ->     Customer_ID INT,
  ->     Old_Balance DECIMAL(10,2),
  ->     New_Balance DECIMAL(10,2),
  ->     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
  -> );
Query OK, 0 rows affected (0.07 sec)
```

RESULT AND JUSTIFICATION:

The trigger automates database operations, maintains data integrity, and reduces manual intervention.

8) A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

SCENARIO UNDERSTANDING:

Frequently performed database operations should be automated to improve efficiency and reduce repetitive SQL coding.

SOLUTION APPROACH:

A stored procedure is created to execute a predefined set of SQL statements whenever required.

```
mysql> CREATE TABLE Employee (  
->     Emp_ID INT PRIMARY KEY,  
->     Emp_Name VARCHAR(50),  
->     Department VARCHAR(30),  
->     Salary DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql>  
mysql> INSERT INTO Employee VALUES  
-> (101, 'Arun', 'HR', 30000),  
-> (102, 'Priya', 'IT', 45000),  
-> (103, 'Rahul', 'Finance', 50000),  
-> (104, 'Meena', 'IT', 40000);  
Query OK, 4 rows affected (0.00 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql>  
mysql> DELIMITER $$  
mysql>  
mysql> CREATE PROCEDURE GetEmployeeByDepartment(IN Dept VARCHAR(30))  
-> BEGIN  
->     SELECT *  
->     FROM Employee  
->     WHERE Department = Dept;  
-> END $$  
Query OK, 0 rows affected (0.02 sec)  
  
mysql>  
mysql> DELIMITER ;  
mysql>  
mysql> CALL GetEmployeeByDepartment('IT');
```

Emp_ID	Emp_Name	Department	Salary
102	Priya	IT	45000.00
104	Meena	IT	40000.00

```
2 rows in set (0.01 sec)  
  
Query OK, 0 rows affected (0.01 sec)  
  
mysql>
```

RESULT AND JUSTIFICATION:

The stored procedure simplifies database operations, improves code reusability, and enhances application performance.

9) For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

SCENARIO UNDERSTANDING:

The database requires customized calculations that can be reused in different SQL queries.

SOLUTION APPROACH:

A user-defined function is developed to perform specific computations and return the required result.

```
mysql>
mysql> CREATE TABLE Student (
  ->     Student_ID INT PRIMARY KEY,
  ->     Student_Name VARCHAR(50),
  ->     Marks INT
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO Student VALUES
  -> (101,'Arun',90),
  -> (102,'Priya',75),
  -> (103,'Rahul',82),
  -> (104,'Meena',65);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION GetGrade(Mark INT)
  -> RETURNS VARCHAR(2)
  -> DETERMINISTIC
  -> BEGIN
  ->     DECLARE Grade VARCHAR(2);
  ->
  ->     IF Mark >= 90 THEN
  ->         SET Grade = 'A';
  ->     ELSEIF Mark >= 75 THEN
  ->         SET Grade = 'B';
  ->     ELSEIF Mark >= 60 THEN
  ->         SET Grade = 'C';
  ->     ELSE
  ->         SET Grade = 'F';
  ->     END IF;
  ->
  ->     RETURN Grade;
  -> END $$
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql>
mysql> SELECT
    ->     Student_ID,
    ->     Student_Name,
    ->     Marks,
    ->     GetGrade(Marks) AS Grade
    -> FROM Student;
```

Student_ID	Student_Name	Marks	Grade
101	Arun	90	A
102	Priya	75	B
103	Rahul	82	B
104	Meena	65	C

```
4 rows in set (0.04 sec)

mysql> |
```

RESULT AND JUSTIFICATION:

The function improves query readability, supports code reuse, and provides consistent results across the database

10) A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

SCENARIO UNDERSTANDING:

The existing SQL queries are inefficient and affect the performance of the database system.

SOLUTION APPROACH:

The queries are optimized using JOINS, indexing, and query restructuring to improve execution speed.

```
mysql> CREATE TABLE Department (  
  ->     Dept_ID INT PRIMARY KEY,  
  ->     Dept_Name VARCHAR(50)  
  -> );  
Query OK, 0 rows affected (0.07 sec)  
  
mysql>  
mysql> INSERT INTO Department VALUES  
  -> (1,'HR'),  
  -> (2,'IT'),  
  -> (3,'Finance');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql>  
mysql> CREATE TABLE Employee (  
  ->     Emp_ID INT PRIMARY KEY,  
  ->     Emp_Name VARCHAR(50),  
  ->     Salary DECIMAL(10,2),  
  ->     Dept_ID INT,  
  ->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
  -> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> INSERT INTO Employee VALUES  
  -> (101,'Arun',30000,1),  
  -> (102,'Priya',45000,2),  
  -> (103,'Rahul',50000,3),  
  -> (104,'Meena',40000,2),  
  -> (105,'Karan',35000,1);  
Query OK, 5 rows affected (0.04 sec)  
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT Emp_Name
-> FROM Employee
-> WHERE Dept_ID =
-> (
->     SELECT Dept_ID
->     FROM Department
->     WHERE Dept_Name = 'IT'
-> );
```

Emp_Name
Priya
Meena

2 rows in set (0.00 sec)

USING JOINS:

```
mysql> SELECT
->     E.Emp_ID,
->     E.Emp_Name,
->     D.Dept_Name,
->     E.Salary
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.Dept_ID = D.Dept_ID
-> WHERE D.Dept_Name = 'IT';
```

Emp_ID	Emp_Name	Dept_Name	Salary
102	Priya	IT	45000.00
104	Meena	IT	40000.00

2 rows in set (0.04 sec)

USING INDEX:

```
mysql> CREATE INDEX idx_Dept_ID
-> ON Employee(Dept_ID);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT
->     E.Emp_ID,
->     E.Emp_Name,
->     D.Dept_Name,
->     E.Salary
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.Dept_ID = D.Dept_ID;
```

Emp_ID	Emp_Name	Dept_Name	Salary
101	Arun	HR	30000.00
105	Karan	HR	35000.00
102	Priya	IT	45000.00
104	Meena	IT	40000.00
103	Rahul	Finance	50000.00

5 rows in set (0.00 sec)

```
mysql> |
```

RESULT AND JUSTIFICATION:

The optimized queries execute faster, reduce resource usage, and improve overall database performance.